

Lab Experiment -1

8-Puzzle Problem

Aim

To develop a Python program that solves the 8-Puzzle problem using the A* Search algorithm.

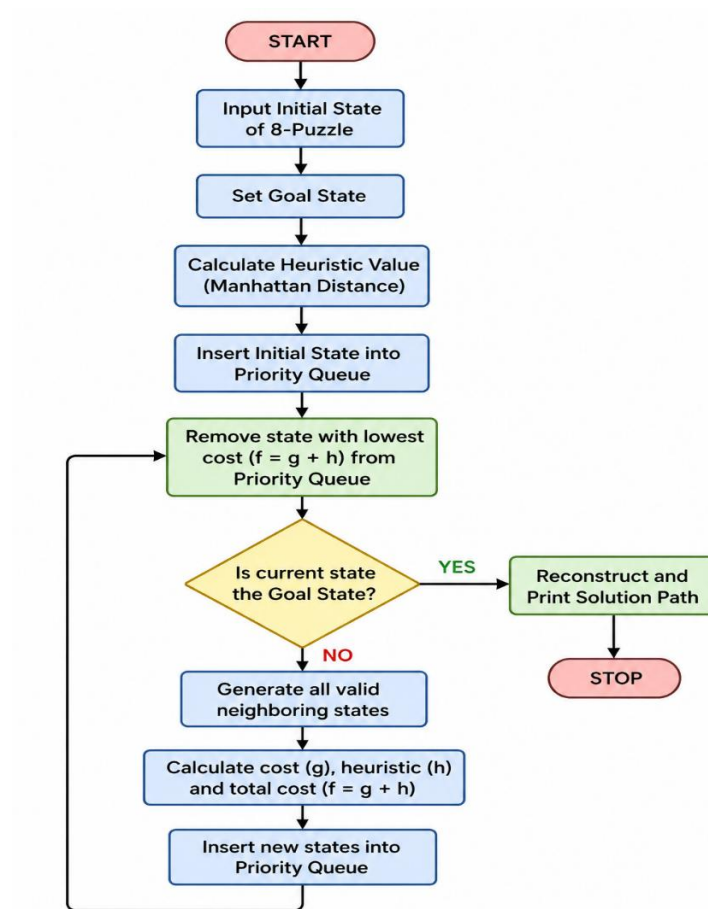
Objective

The objective of this experiment is to implement the A* search algorithm in Python to solve the 8-puzzle problem. It aims to find the optimal sequence of moves from the initial state to the goal state efficiently.

Algorithm

1. Define the initial state and goal state.
2. Calculate the Manhattan Distance heuristic.
3. Store the initial state in a priority queue.
4. Repeat until the goal is found:
 - Remove the state with the lowest cost.
 - If it is the goal state, stop.
 - Generate all possible neighboring states.
 - Calculate their heuristic values.
 - Insert them into the priority queue.
5. Print the solution path.

Flowchart



Code

```
import heapq

GOAL = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

def heuristic(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
```

```

        goal_x = (value - 1) // 3
        goal_y = (value - 1) % 3
        distance += abs(i - goal_x) + abs(j - goal_y)

    return distance

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    directions = [(-1,0),(1,0),(0,-1),(0,1)]
    for dx, dy in directions:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny],
            new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def print_state(state):
    for row in state:
        print(row)

    print()

def a_star(start):

```

```

pq = []
heapq.heappush(pq, (heuristic(start), 0, start, []))
visited = set()
while pq:
    f, g, current, path = heapq.heappop(pq)
    if current in visited:
        continue
    visited.add(current)
    if current == GOAL:
        return path + [current]
    for neighbor in get_neighbors(current):
        if neighbor not in visited:
            heapq.heappush(
                pq,
                (g + 1 + heuristic(neighbor),
                 g + 1,
                 neighbor,
                 path + [current])
            )
    return None
start = (
    (1,2,3),
    (4,0,6),
    (7,5,8)
)
solution = a_star(start)

```

```

if solution:

    print("Solution found in", len(solution)-1, "moves\n")

    for step in solution:

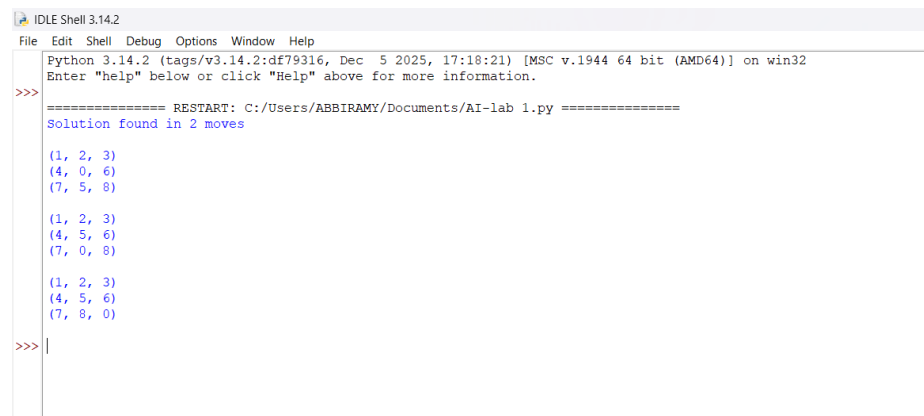
        print_state(step)

else:

    print("No solution exists.")

```

Output



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ABBIRAMY/Documents/AI-lab 1.py =====
Solution found in 2 moves

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
>>> |

```

Result

The Python program successfully solved the given 8-Puzzle problem using the A* search algorithm and generated the shortest path to reach the goal state.

Conclusion

The A* search algorithm efficiently solves the 8-Puzzle problem by using the Manhattan Distance heuristic. It explores the most promising states first, reducing the search time and producing the optimal solution.